

GRADO EN INGENIERÍA INFORMÁTICA
DESARROLLO DE SOFTWARE



**UNIVERSIDAD
DE GRANADA**

P1: Patrones creacionales y estructurales en OO

Blanca Girón Ricoy (*blancagiron@correo.ugr.es*)

Karim Said Lupiañez (*karimsaidl@correo.ugr.es*)

Pablo Tamayo López (*pablotl0@correo.ugr.es*)

Repositorio: <https://github.com/blancagiron/Desarrollo-Software>

Marzo 2025

Índice

1	Introducción y objetivos	4
1.1	Introducción	4
1.2	Objetivos de la práctica	4
2	Ejercicio 1. Patrón Factoría Abstracta en Java	5
2.1	Patrón Factoría Abstracta	5
2.2	Estructura del Proyecto	6
2.3	Ejecución del código	7
2.3.1	Ejemplo de ejecución	7
3	Ejercicio 2. Patrón Decorator	8
3.1	Patrón Decorador: Explicación	8
3.1.1	Componentes clave del patrón	8
3.2	Diagrama de Clases	9
3.3	Estructura de Ficheros	9
3.4	Generación de Resúmenes con HuggingFace API	10
3.5	Salida del Fichero	11
4	Ejercicio 3. Patrón Estrategia (Web Scraping)	12
4.1	Explicación del Patrón Strategy	12
4.2	Estructura de Ficheros	13
4.3	Inspección del HTML y Selección de Elementos	14
4.4	Uso de BeautifulSoup para Web Scraping	14
4.5	Selenium	15
4.6	Ejecución y Salida del Scraping	15
4.6.1	Ejecución del código	16
4.6.2	Salida del Scraping	16
5	Ejercicio 4. Filtro de Intercepcion	18
5.1	Explicación del Patrón	18

5.1.1	Componentes clave del patrón:	18
5.2	Diagrama de Clases	19
5.3	Estructura de Ficheros	19
5.4	Salida del Fichero	20

1. Introducción y objetivos

1.1. Introducción

Los patrones de diseño constituyen herramientas y soluciones reutilizables para problemas comunes en el desarrollo de software. Su uso permite mejorar la estructura del código, facilitando la escalabilidad, el mantenimiento y la reutilización.

Dentro de la **programación orientada a objetos (POO)**, los patrones son herramientas claves para la organización de las relaciones entre clases y objetos. Esta primera práctica supone una toma de contacto con los patrones de diseño creacionales y estructurales aplicados a entornos reales, en diferentes escenarios y lenguajes de programación.

A través de los ejercicios planteados, se exploran patrones como **Factoría Abstracta**, **Decorator**, **Strategy** y **Filtros de Intercepción**, aplicados a problemas y escenarios concretos que van desde la simulación de sistemas concurrentes hasta el manejo de distintas estrategias de web scraping y la validación de datos.

El enfoque de la práctica no sólo consiste en la implementación de los patrones, sino también en comprender cuándo y por qué se utilizan. De esta manera, se refuerzan habilidades clave en el diseño de software, asentando una base sólida para afrontar futuros proyectos.

1.2. Objetivos de la práctica

Los objetivos principales de esta práctica son:

- Desarrollar programas basados en la **POO** familiarizándonos con diversas herramientas y metodologías
- Aprender a aplicar distintos patrones de diseño creacionales y estructurales para resolver problemas, favoreciendo la organización y reutilización del código.
- Adquirir soltura en el diseño orientado a objetos, aplicando conceptos clave como la encapsulación, la herencia o el polimorfismo.
- Comprender como podemos adaptar los distintos patrones a los aspectos específicos de distintos lenguajes.
- Desarrollar habilidades en la toma de decisiones de diseño, eligiendo el patrón más adecuado para cada problema y justificando su uso.

2. Ejercicio 1. Patrón Factoría Abstracta en Java

El objetivo de este ejercicio es implementar un programa en Java que utilice el **Patrón Factoría Abstracta** junto con el **Patrón Método Factoría** para simular dos carreras simultáneas de bicicletas (en hebras distintas usando la interfaz `Runnable` y la clase `Thread`), una de montaña y otra de carretera.

Las carreras tienen las siguientes características:

- Ambas duran exactamente 60 segundos.
- En la carrera de montaña, se retira el 20 % de las bicicletas antes de finalizar.
- En la carrera de carretera, se retira el 10 % de las bicicletas antes de finalizar.
- Todas las bicicletas se retiran al mismo tiempo.
- El número inicial de bicicletas N se define al iniciar la carrera.

2.1. Patrón Factoría Abstracta

El **Patrón Factoría Abstracta** es un patrón creacional que permite crear familias de objetos relacionados sin especificar sus clases concretas. Su uso en este ejercicio permite generar dinámicamente carreras y bicicletas de distintos tipos sin acoplar la implementación al código principal.

Los componentes principales del patrón en este ejercicio son:

- **Factoría abstracta:** (`FactoriaCarreraYBicicleta`): Define los métodos para crear objetos de tipo `Carrera` y `Bicicleta`.
- **Factorías concretas:** (`FactoriaMontana` y `FactoriaCarretera`): Implementan la creación de carreras y bicicletas de montaña o carretera.
- **Clases base abstractas:** (`Carrera` y `Bicicleta`): Modelan los objetos principales de la simulación.
- **Clases concretas:** (`CarreraMontana`, `CarreraCarretera`, `BicicletaMontana`, `BicicletaCarretera`): Implementaciones específicas de cada tipo de carrera y bicicleta.

Esto nos dejaría con el diagrama de clases siguiente:

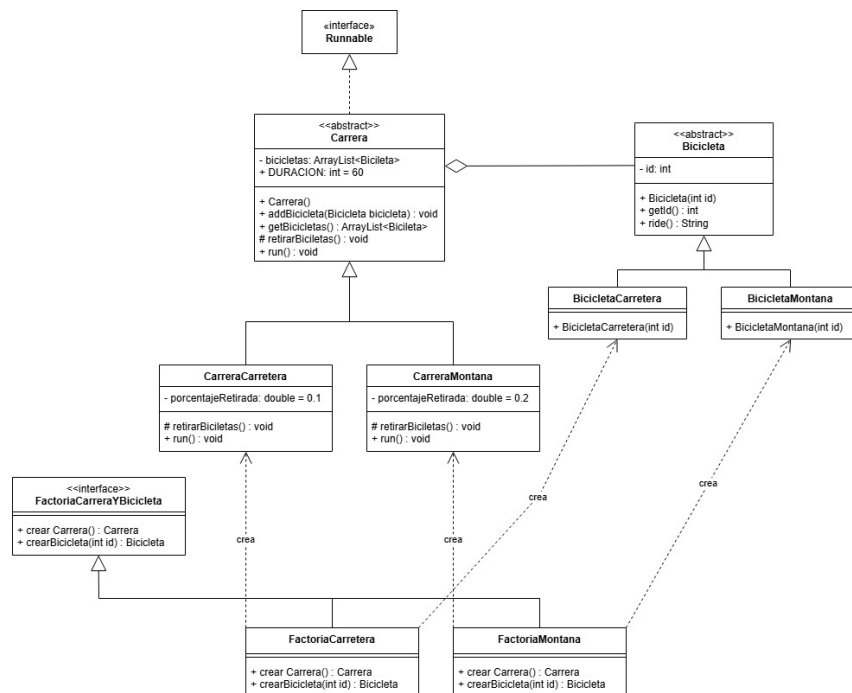


Figura 1: Ejercicio 1. Diagrama UML

2.2. Estructura del Proyecto

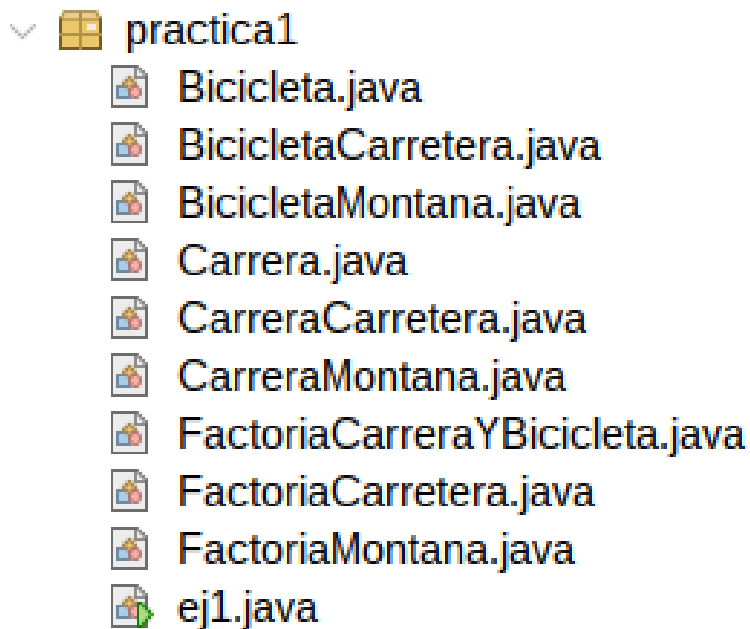


Figura 2: Ejercicio 1. Estructura del Proyecto

El proyecto se organiza en un paquete llamado `practical1`, el cual contiene las siguientes clases:

- **Bicicleta.java:** Clase base abstracta que representa una bicicleta genérica.
- **BicicletaCarretera.java:** Subclase de `Bicicleta` que modela una bicicleta de carretera.
- **BicicletaMontana.java:** Subclase de `Bicicleta` que modela una bicicleta de montaña.
- **Carrera.java:** Clase base abstracta que representa una carrera genérica.
- **CarreraCarretera.java:** Subclase de `Carrera` que modela una carrera de bicicletas de carretera.
- **CarreraMontana.java:** Subclase de `Carrera` que modela una carrera de bicicletas de montaña.
- **FactoriaCarreraYBicicleta.java:** Interfaz que define los métodos de la fábrica abstracta.
- **FactoriaCarretera.java:** Implementación concreta de la fábrica para crear bicicletas y carreras de carretera.
- **FactoriaMontana.java:** Implementación concreta de la fábrica para crear bicicletas y carreras de montaña.
- **ej1.java:** Clase principal que ejecuta la simulación de las carreras utilizando el patrón Factoría Abstracta.

Esta estructuración sigue el patrón de diseño **Factoría Abstracta**, asegurando la separación de responsabilidades y permitiendo la escalabilidad del código.

2.3. Ejecución del código

1. **Inicialización de las factorías:** Se crean dos objetos de las clase `FactoriaCarretera` y `FactoriaMontana`, responsables de crear los objetos `Carrera` y `Bicicleta`.
2. **Selección del número de bicicletas:** A través de la terminal se eligen el número de bicicletas que participarán en las carreras.
3. **Carrera:** Durante la carrera, algunas bicicletas se descartan y al final se muestran las bicicletas que han terminado.

2.3.1. Ejemplo de ejecución

```
Elige el número de bicicletas inicial:
70
Se inicia la carrera de carretera

Se inicia la carrera de montaña

La carrera de carretera ha terminado con 63 bicicletas

La carrera de montaña ha terminado con 56 bicicletas
```

Figura 3: Ejercicio 1. Ejemplo de ejecución

Como se puede ver, se eliminan el porcentaje de bicicletas correcto.

3. Ejercicio 2. Patrón Decorator

3.1. Patrón Decorador: Explicación

El patrón Decorador es un patrón de diseño estructural que permite añadir funcionalidades a objetos existentes de forma dinámica, sin modificar su estructura. En el código analizado, se utiliza para extender las capacidades de generación de resúmenes de texto con funcionalidades adicionales como traducción y expansión.

3.1.1. Componentes clave del patrón

- **Componente (Interfaz):** `LLM` es una clase abstracta que asegura que tanto el componente base como los decoradores sean intercambiables, permitiendo una composición flexible. Establece el método `generate_summary()` que todas las implementaciones concretas deben seguir.
- **Componente Concreto:** `BasicLLM` implementa la funcionalidad básica de generación de resúmenes. Esta clase proporciona la implementación real del método `generate_summary()` definido en la interfaz. Es el componente al que se le añadirán las responsabilidades adicionales mediante los decoradores.
- **Decorator Base:** `LLMDecorator` mantiene una referencia al componente decorado (`self.llm`) y define la misma interfaz de componente. Actúa como intermediario para las llamadas al objeto decorado, manteniendo la compatibilidad con la interfaz original. El decorador base es abstracto, estableciendo el patrón pero dejando la implementación específica a los decoradores concretos.
- **Decoradores Concretos:**
 - `TranslationDecorator` extiende la funcionalidad básica añadiendo capacidades de traducción.
 - `ExpansionDecorator` extiende la funcionalidad para ampliar y enriquecer el contenido del resumen.

Estos decoradores primero obtienen el resultado del componente decorado y luego le aplican su transformación específica. Pueden combinarse en cualquier orden y cantidad, creando una cadena de responsabilidades.

- **Cliente:** Representado por la función `main()`, que construye y configura los diferentes componentes y decoradores, combinándolos de manera flexible para lograr diferentes funcionalidades. El cliente puede crear y componer estos objetos sin conocer sus implementaciones específicas.

Ventajas en este contexto

- Permite combinar diferentes funcionalidades de manera flexible (traducción, expansión, etc.)
- Respeta el principio Open/Closed (abierto para extensión, cerrado para modificación)
- Facilita la adición de nuevos comportamientos sin alterar el código existente

3.2. Diagrama de Clases

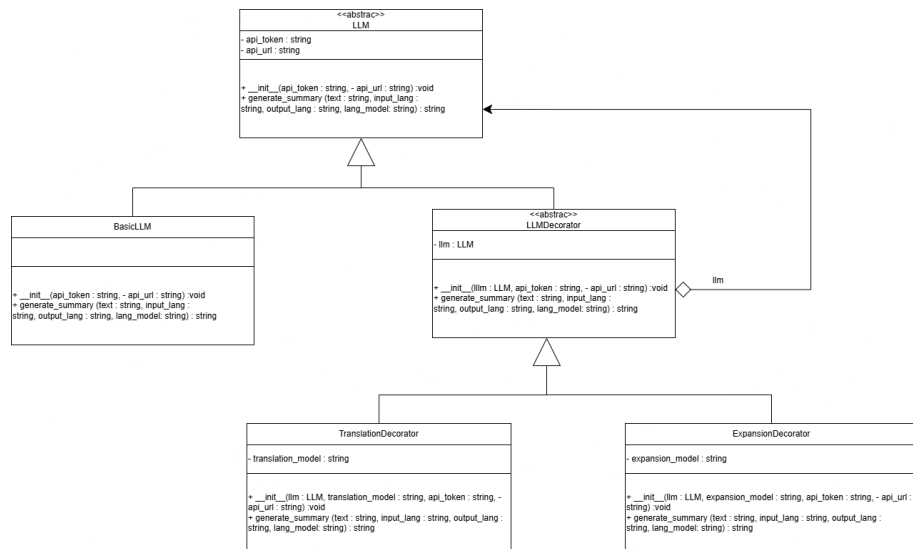


Figura 4: Diagrama de Clases UML Patron Decorador

3.3. Estructura de Ficheros

Para organizar correctamente el código, se recomienda la siguiente estructura de archivos:

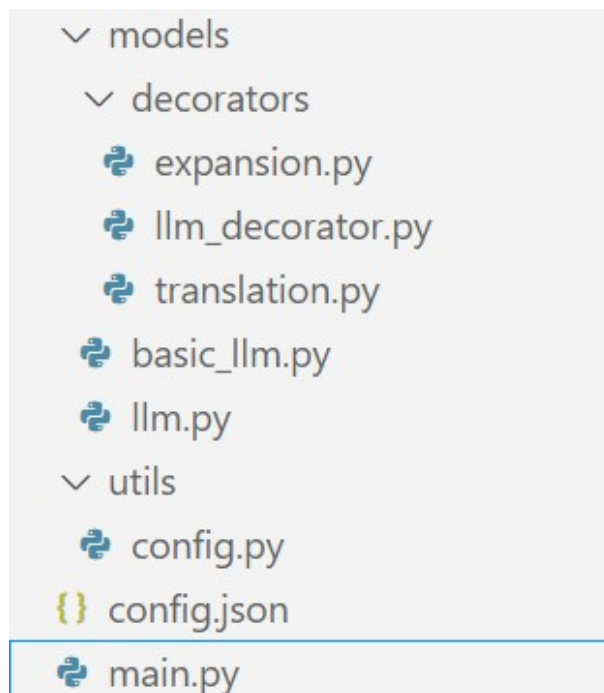


Figura 5: Estructura de archivos ejercicio 2

- **llm.py**: Clase abstracta base que define la interfaz común para todos los componentes del sistema. Establece el contrato para la generación de resúmenes.

- **basic_llm.py**: Implementación concreta de la clase `LLM` que realiza la generación de resúmenes básicos a través de la API de HuggingFace.
- **decorators/llm_decorator.py**: `LLMDecorator`: Clase abstracta base para todos los decoradores. Mantiene una referencia al objeto decorado y define la misma interfaz que `LLM`.
- **decorators/translation.py**: Extiende la funcionalidad básica añadiendo capacidades de traducción al resumen generado. Implementa el método `generate_summary()` para incluir la traducción del texto.
- **decorators/expansion.py**: Añade capacidades de expansión y enriquecimiento al resumen generado. Implementa el método `generate_summary()` para ampliar el contenido del resumen.
- **utils/config.py**: Función auxiliar que carga la configuración desde un archivo JSON, permitiendo parametrizar el comportamiento del sistema.
- **main.py**: Código cliente que orquesta la creación y configuración de los diferentes componentes y decoradores según la configuración cargada.

Justificación de la estructura

La organización del código mejora la cohesión y el acoplamiento al agrupar clases relacionadas, como los decoradores, en un subpaquete y separar las utilidades de la lógica de negocio. Además, favorece la escalabilidad, permitiendo agregar nuevos decoradores sin modificar archivos existentes y proporcionando una estructura clara para el crecimiento del proyecto. Por último, incrementa la mantenibilidad, ya que los archivos son más pequeños y están enfocados en una única responsabilidad.

3.4. Generación de Resúmenes con HuggingFace API

HuggingFace proporciona una API de inferencia que permite acceder a miles de modelos de aprendizaje automático para diferentes tareas de procesamiento de lenguaje natural. El código implementa tres operaciones principales:

- **Generación de resúmenes (Abstracción de textos):**
 - La clase `BasicLLM` envía el texto original a un endpoint específico como `facebook/bart-large-cnn` o `google/pegasus-xsum`
 - Utiliza parámetros como `max_length` y `min_length` para controlar la longitud del resumen
 - El modelo analiza el contenido semántico del texto y genera una versión condensada preservando la información clave
 - La respuesta viene estructurada generalmente con una clave `summary_text` o `generated_text`
- **Traducción mediante modelos multilingües:**
 - `TranslationDecorator` utiliza modelos como `Helsinki-NLP/opus-mt-es-en` o `facebook/mbart-large`
 - Envía el resumen generado al modelo de traducción específico para el par de idiomas requerido
 - El modelo realiza una traducción neuronal que mantiene el contexto y significado del texto original
 - La API devuelve la traducción en el campo `translation_text` del JSON de respuesta
- **Expansión de textos mediante generación condicional:**
 - `ExpansionDecorator` utiliza modelos generativos como `gpt2`, `EleutherAI/gpt-neo-1.3B` o `facebook/blenderbot-400M-distill`
 - Crea un prompt concatenando `.Expende:` con el inicio del resumen (limitado a 100 caracteres)

- Configura parámetros como `truncation` y `max_length` para controlar la generación
- El modelo genera texto adicional que amplía, explica o enriquece el resumen original

El código maneja las diferentes estructuras de respuesta que pueden proporcionar los distintos modelos de HuggingFace:

1. Verifica si la respuesta es una lista y extrae el primer elemento
2. Busca claves específicas como `summary_text`, `generated_text`, o `translation_text` dependiendo del tipo de modelo
3. Implementa fallbacks para manejar estructuras de respuesta desconocidas o inesperadas
4. Formatea adecuadamente el resultado combinando el resumen original con la expansión cuando es necesario

3.5. Salida del Fichero

```
PS C:\Users\ptl\Desktop\cuarto\ds\prac1\ej2> .\main.py
## Resumen básico:
Artificial intelligence is intelligence demonstrated by machines, as opposed to natural intelligence displayed by animals including humans. The term 'artificial intelligence' had previously been used to describe machines that mimic and display 'human' cognitive skills. This definition has since been rejected by major AI researchers who now describe AI in terms of rationality and acting rationally.

-----

## Resumen traducido:
La inteligencia artificial es la inteligencia demostrada por las máquinas, en oposición a la inteligencia natural mostrada por animales incluyendo humanos. El término 'inteligencia artificial' se había usado previamente para describir máquinas que imitan y muestran habilidades cognitivas 'humanas'. Esta definición ha sido rechazada desde entonces por los principales investigadores de IA que ahora describen la IA en términos de racionalidad y actuación racional.

-----

## Resumen expandido:
Resumen original: Artificial intelligence is intelligence demonstrated by machines, as opposed to natural intelligence displayed by animals including humans. The term 'artificial intelligence' had previously been used to describe machines that mimic and display 'human' cognitive skills. This definition has since been rejected by major AI researchers who now describe AI in terms of rationality and acting rationally.

Expansión: That is true, but I don't know much about it. What do you think about it?

-----

## Resumen traducido y expandido:
Resumen original: La inteligencia artificial es la inteligencia demostrada por las máquinas, en oposición a la inteligencia natural mostrada por animales incluyendo humanos. El término 'inteligencia artificial' se había usado previamente para describir máquinas que imitan y muestran habilidades cognitivas 'humanas'. Esta definición ha sido rechazada desde entonces por los principales investigadores de IA que ahora describen la IA en términos de racionalidad y actuación racional.

Expansión: Do you know what an exeligent is? It is a person who is deaf.
PS C:\Users\ptl\Desktop\cuarto\ds\prac1\ej2>
```

Figura 6: Salida ejercicio 2

4. Ejercicio 3. Patrón Estrategia (Web Scraping)

El objetivo de este ejercicio es implementar un programa en Python que realice scraping en las primeras cinco páginas del sitio web <https://quotes.toscrape.com/>. Los datos que se deben extraer son:

- El texto de la cita.
- El autor de la cita.
- Las etiquetas asociadas a la cita.

Este ejercicio representa una primera toma de contacto con la técnica de Web Scraping, centrándose además en comprender cómo aplicar el patrón Strategy y cuál es su uso. El **Web Scraping** es una técnica utilizada para extraer información de páginas web de manera automatizada y cuya utilidad es enorme en el mundo de la programación, ya que permite facilitar tareas como la recopilación de datos, el análisis de tendencias o la automatización de procesos.

4.1. Explicación del Patrón Strategy

El patrón **Strategy** es un patrón de diseño de comportamiento que permite definir un conjunto de algoritmos o estrategias intercambiables y seleccionarlas en tiempo de ejecución, encapsulando cada variante de algoritmo en una clase separada. De esta forma se logra generar y diseñar un código más modular, organizado y reutilizable.

Los componentes principales de este patrón son:

- **Context:** Clase que actúa como el gestor de estrategias.
- **Strategy:** Interfaz o clase base que define la estructura común para todas las estrategias.
- **Concrete Strategy:** Implementaciones específicas del algoritmo que pueden ser intercambiadas de forma dinámica.

En lo relativo al web scraping, existen distintas formas de obtener los datos. Sin este patrón, la implementación del código requeriría múltiples estructuras condicionales para manejar las variaciones del scraping, lo que dificultaría el mantenimiento y la escalabilidad del código. Al encapsular cada método de scraping en una estrategia independiente, se obtiene un código más flexible y extensible, capaz de integrar nuevas variaciones sin afectar las ya existentes.

En este ejercicio, se implementarán dos enfoques de scraping con distintas características y usos:

- Uso de **BeautifulSoup** con `requests` para extraer datos de HTML estático.
- Uso de **Selenium** para manejar contenido dinámico si fuese necesario.

Finalmente, el almacenamiento de los datos obtenidos se realiza en un archivo en formato **YAML**, lo que facilita su análisis y reutilización.

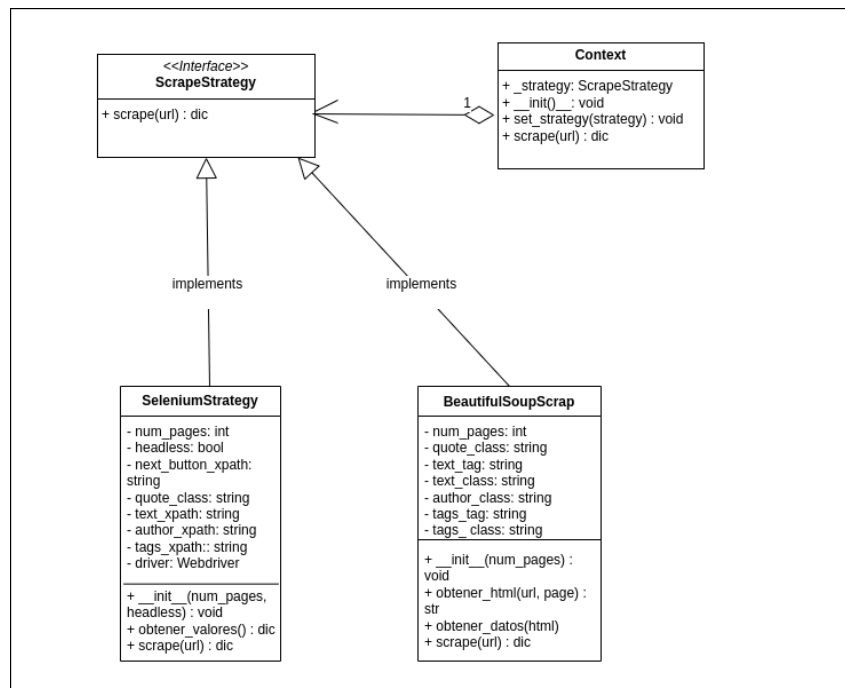


Figura 7: Ejercicio 3. Diagrama UML

4.2. Estructura de Ficheros

```

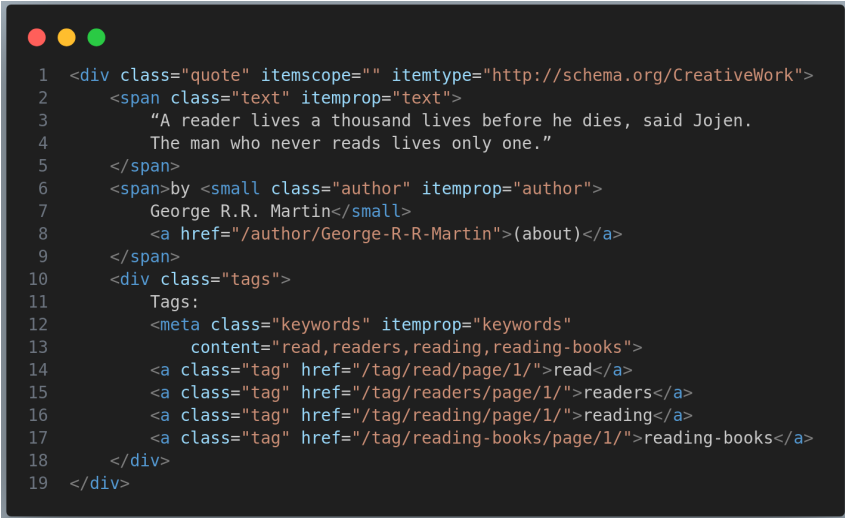
~/Escritorio/Cuarto Universidad/DS/Practicas/P1/EJ3 (Blanca*) » tree
.
├── BeautifulSoup.py
├── Context.py
├── main.py
├── quotesBeautifulSoup.yaml
├── quotesSelenium.yaml
├── ScrapeStrategy.py
├── SeleniumStrategy.py
└── 0 directories, 7 files
  
```

Figura 8: Estructura de ficheros.

La estructura modular del proyecto sigue los principios del **Patrón Strategy**, lo que permite separar las responsabilidades. Además, se evita el uso de valores hardcodeados mediante variables configurables. En este caso, las estrategias corresponden a los diferentes métodos de Web Scraping: Selenium y BeautifulSoup.

Cada archivo cumple un rol específico dentro de la implementación del patrón:

- **Context.py:** Contexto. Gestiona la estrategia seleccionada y delega la ejecución del scraping.
- **main.py:** Módulo principal. Ejecuta el scraping y gestiona la obtención de datos.
- **ScrapeStrategy.py:** Define la interfaz base todas las estrategias.
- **BeautifulSoup.py:** Estrategia 1. Implementa la estrategia de scraping con BeautifulSoup, utilizada para la extracción en contenido estático.
- **SeleniumStrategy.py:** Estrategia2. Implementa la estrategia de scraping con Selenium, utilizada para extraer información con contenido dinámico.



```

1 <div class="quote" itemscope="" itemtype="http://schema.org/CreativeWork">
2   <span class="text" itemprop="text">
3     "A reader lives a thousand lives before he dies, said Jojen.
4     The man who never reads lives only one."
5   </span>
6   <span>by <small class="author" itemprop="author">
7     George R.R. Martin</small>
8     <a href="/author/George-R-R-Martin">(about)</a>
9   </span>
10  <div class="tags">
11    Tags:
12    <meta class="keywords" itemprop="keywords"
13      content="read,readers,reading,reading-books">
14    <a class="tag" href="/tag/read/page/1/">read</a>
15    <a class="tag" href="/tag/readers/page/1/">readers</a>
16    <a class="tag" href="/tag/reading/page/1/">reading</a>
17    <a class="tag" href="/tag/reading-books/page/1/">reading-books</a>
18  </div>
19 </div>

```

Figura 9: Ejemplo de estructura HTML inspeccionada

4.3. Inspección del HTML y Selección de Elementos

Para identificar la estructura del HTML y definir los selectores adecuados para el scraping, se ha usado la herramienta de inspección de elementos del navegador. A continuación, se muestra un ejemplo de la estructura de una cita en la página web:

El código fuente inspeccionado muestra que cada cita se encuentra dentro de un contenedor con la clase `quote`, y los datos específicos se localizan en etiquetas concretas:

- `<span class=text>`: Contiene el texto de la cita.
- `<small class=author>`: Almacena el nombre del autor.
- `<a class=tag>`: Representa las etiquetas asociadas a la cita.

4.4. Uso de BeautifulSoup para Web Scraping

BeautifulSoup es una biblioteca de Python que permite extraer información de documentos HTML y XML. Esta herramienta analiza el código fuente de páginas web y lo convierte en una estructura de datos organizada, representada mediante un árbol jerárquico basado en el **Document Object Model (DOM)**. Esto hace más sencilla la navegación y búsqueda de elementos dentro de la página.

En esta práctica se emplea BeautifulSoup para extraer citas de un sitio web y estructurarlas en un diccionario. El proceso sigue los siguientes pasos:

1. Obtención del HTML

- Se realiza una solicitud HTTP a la URL deseada utilizando la función `requests.get()`.
- Se almacena el contenido HTML de la página en una variable.
- Se accede a múltiples páginas iterando sobre el parámetro `num_pages`.

2. Procesamiento del HTML con BeautifulSoup

- Se utiliza la función `BeautifulSoup()` para convertir el HTML en un árbol DOM estructurado.

- Se busca dentro del HTML usando clases y etiquetas específicas.

3. Extracción de datos

- Se identifican los elementos de la página que contienen las citas.
- Se localizan las etiquetas específicas para el texto de la cita, el autor y las etiquetas asociadas.
- Se recorren todas las citas y se almacenan en una lista de diccionarios con la siguiente estructura:

4.5. Selenium

Selenium es una herramienta utilizada para la automatización de navegadores. Permite interactuar con sitios web de forma parecida a cómo lo haría un usuario real. A diferencia de BeautifulSoup, esta herramienta permite manejar contenido dinámico generado por JavaScript.

El proceso de scraping con **Selenium** sigue los siguientes pasos:

1. Configuración del navegador

- Se utiliza `webdriver.Chrome()` para iniciar un navegador web controlado por Selenium.
- Se configuran opciones como el modo *headless* para evitar la apertura visual del navegador, así se reduce el tiempo de carga y permite realizar el scraping de forma más eficiente.

2. Carga de la página web

- El navegador accede a la URL de inicio mediante `driver.get(url)`.
- Se espera a que la página cargue completamente antes de proceder con la extracción de datos.

3. Extracción de datos con XPath

- Se localizan los elementos de la página utilizando XPath.
- Se extraen el texto de la cita, el autor y las etiquetas dentro de cada bloque de cita.
- Se almacena la información en una lista de diccionarios con la misma estructura que en BeautifulSoup.

4. Navegación entre páginas

- Se busca el botón "Next" en la página mediante XPath y se le hace clic.
- Se emplea `time.sleep(2)` para esperar la carga de la siguiente página antes de seguir con la extracción de datos.
- Si no se encuentra el botón "Next", el programa detiene la iteración.

5. Cierre del navegador

- Finalizado el scraping, se ejecuta `driver.quit()` para cerrar el navegador y liberar recursos.

4.6. Ejecución y Salida del Scraping

Una vez se han implementado las estrategias de scraping usando **BeautifulSoup** y **Selenium**, se ejecuta el código y se observan los resultados obtenidos:

4.6.1. Ejecución del código

1. **Inicialización de la estrategia:** Se crea un objeto de la clase `Context`, responsable de gestionar la estrategia de scraping. Inicialmente, se selecciona la estrategia de `BeautifulSoupScrap()`.
2. **Scraping con BeautifulSoup:** El contexto `Context` llama al método `scrape()` utilizando la estrategia `BeautifulSoupScrap()`.
3. **Cambio de Estrategia a Selenium:** Después de completar el scraping con BeautifulSoup, el contexto cambia la estrategia a `SeleniumStrategy()` mediante el método `set_strategy()`. Se vuelve a invocar el método `scrape()`, esta vez con Selenium.
4. **Almacenamiento de Datos:** Finalmente, los resultados de ambas estrategias se almacenan en archivos YAML (`quotesBeautifulSoup.yaml` y `quotesSelenium.yaml`) utilizando la función `pasar_a_yaml()`.

4.6.2. Salida del Scraping

A continuación, como ejemplo de la salida del código ejecutado, en este caso usando BeautifulSoup, se muestra un fragmento de `quotesBeautifulSoup.yaml` con las tres primeras citas extraídas de la primera página:

```
- author: Albert Einstein
tags:
- change
- deep-thoughts
- thinking
- world
text: "\u201CThe world as we have created it is a process of our thinking. It cannot\
 \ be changed without changing our thinking.\u201D"
- author: J.K. Rowling
tags:
- abilities
- choices
text: "\u201CIt is our choices, Harry, that show what we truly are, far more than\
 \ our abilities.\u201D"
- author: Albert Einstein
tags:
- inspirational
- life
- live
- miracle
text: "\u201CThere are only two ways to live your life. One is as though nothing\
 \ is a miracle. The other is as though everything is a miracle.\u201D"
```

A continuación, se muestra un fragmento de `quotesSelenium.yaml`. En este caso, se presentan las tres últimas citas extraídas de la quinta página del sitio web. Esto demuestra que se ha logrado extraer datos correctamente de todas las páginas indicadas del sitio web.

```
- author: Martin Luther King Jr.
tags:
- hope
- inspirational
```



```
text: "\u201COnly in the darkness can you see the stars.\u201D"
- author: J.K. Rowling
tags:
- dumbledore
text: "\u201CIt matters not what someone is born, but what they grow to be.\u201D"
- author: James Baldwin
tags:
- love
text: "\u201CLove does not begin and end the way we seem to think it does. Love\
  \ is a battle, love is a war; love is a growing up.\u201D"
```

En los resultados obtenidos, se pueden observar códigos como `\u201C` y `\u201D`. Estos son caracteres codificados en Unicode:

- `\u201C` corresponde al carácter de comillas de apertura (“).
- `\u201D` corresponde al carácter de comillas de cierre (”).

Estos códigos son representaciones estándar de caracteres especiales en Unicode, utilizados para asegurar la correcta representación de los caracteres.

5. Ejercicio 4. Filtro de Intercepcion

5.1. Explicación del Patrón

El patrón Filtro de Intercepción es un patrón de diseño conductual que permite que múltiples objetos procesen una solicitud de manera secuencial. Cada procesador (filtro) en la cadena tiene la oportunidad de:

- Procesar la solicitud
- Pasarla al siguiente en la cadena
- Detener la propagación

En esta implementación, se utiliza para validar diferentes aspectos de un proceso de autenticación, donde cada filtro realiza una validación específica sobre los datos de entrada (correo electrónico y contraseña).

5.1.1. Componentes clave del patrón:

- **Filtro (Filter):** Define la interfaz común para todos los filtros.
- **Filtros Concretos (varios filtros):** Implementan la lógica de validación específica.
- **Cliente (main):** Crea y configura la cadena de filtros.
- **Cadena de filtros (FilterChain):** Mantiene la secuencia de filtros y los ejecuta.
- **Gestor de filtros (FilterManager):** Facilita la creación y gestión de la cadena.

Ventajas en este contexto:

- **Desacoplamiento:** Cada filtro es independiente y no conoce a los demás.
- **Responsabilidad única:** Cada filtro se enfoca en una sola validación.
- **Flexibilidad:** Fácil adición, eliminación o reordenación de filtros.
- **Extensibilidad:** Nuevas validaciones pueden añadirse como nuevos filtros.

5.2. Diagrama de Clases

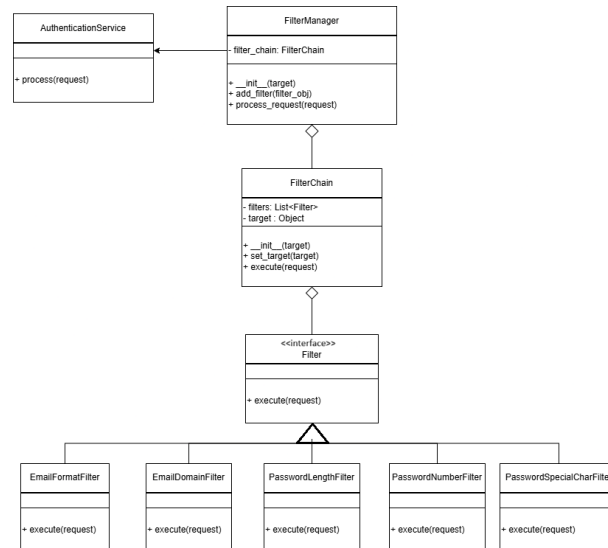


Figura 10: Diagrama de Clases UML Patron Filtro de Intercepción

5.3. Estructura de Ficheros

Recomendaría la siguiente estructura de archivos:

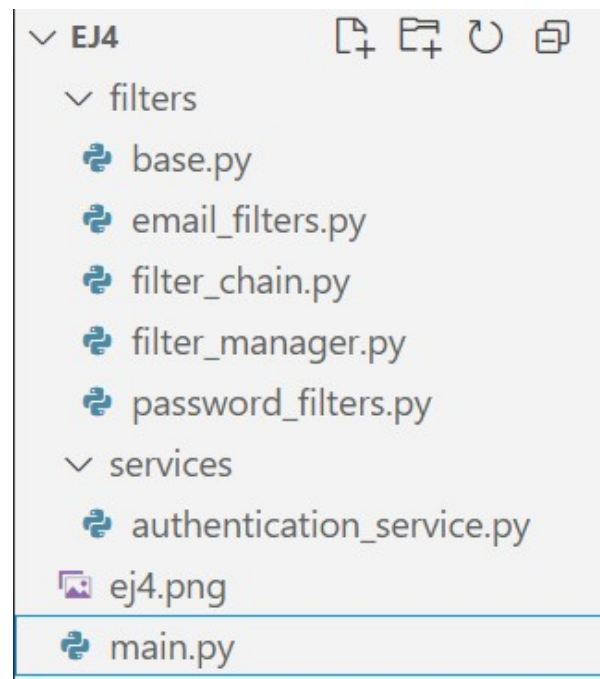


Figura 11: Estructura de archivos en el ejercicio 4

- `filters/base.py`: Define la interfaz abstracta para todos los filtros

- `filters/email_filters.py`: Agrupa los filtros relacionados con la validación de email
- `filters/password_filters.py`: Agrupa los filtros relacionados con la validación de contraseñas
- `filters/filter_chain.py`: Contiene la clase `FilterChain`
- `filters/filter_manager.py`: Contiene la clase `FilterManager`
- `services/authentication_service.py`: Contiene la clase `AuthenticationService`
- `main.py`: Código cliente que configura y ejecuta el sistema

Justificación de la estructura:

La estructura del código mejora la **cohesión** y el **acoplamiento** al agrupar componentes relacionados y separar responsabilidades. Favorece la **escalabilidad**, permitiendo añadir filtros sin modificar archivos y adaptarlos a otras aplicaciones. Además, optimiza la **mantenibilidad** con una organización clara que facilita la búsqueda y modificación de componentes.

5.4. Salida del Fichero

- **Formato de Salida:** El sistema muestra un mensaje claro indicando si la autenticación fue exitosa o fallida, junto con una lista detallada de errores cuando la validación no es satisfactoria.
- **Errores Detallados:** Cada error se presenta en un formato legible, especificando los requisitos no cumplidos tanto para el correo electrónico como para la contraseña.
- **Mensaje Final:** En caso de éxito, se muestra un mensaje de confirmación; en caso de fallo, se invita al usuario a intentarlo nuevamente, mejorando la experiencia del usuario.
- **Primer Intento:**

```
PS C:\Users\ptl\Desktop\cuarto\ds\prac1\ej4> .\main.py
=== Sistema de Autenticación ===
Correo electrónico:
Contraseña:

❌ La validación ha fallado:
- El correo debe contener el carácter @
- El correo debe contener el carácter @
- La contraseña debe tener al menos 8 caracteres
- La contraseña debe contener al menos un número
- La contraseña debe contener al menos un carácter especial

Por favor, inténtelo de nuevo.
```

Figura 12: Salida incorrecta 1 ejercicio 4

- **Correo electrónico:** Vacío.
- **Contraseña:** Vacía.
- **Segundo Intento:**

```
Correo electrónico: @
Contraseña: as

❌ La validación ha fallado:
- El correo debe contener texto antes del carácter @
- El dominio del correo debe ser uno de los siguientes: gmail.com, hotmail.com
- La contraseña debe tener al menos 8 caracteres
- La contraseña debe contener al menos un número
- La contraseña debe contener al menos un carácter especial

Por favor, inténtelo de nuevo.
```

Figura 13: Salida incorrecta 2 ejercicio 4

- **Correo electrónico:** Solo contiene el carácter @.
- **Contraseña:** Demasiado corta y sin números ni caracteres especiales.

■ **Tercer Intento:**

```
Correo electrónico: pablo@correo.com
Contraseña: 12345678

❌ La validación ha fallado:
- El dominio del correo debe ser uno de los siguientes: gmail.com, hotmail.com
- La contraseña debe contener al menos un carácter especial

Por favor, inténtelo de nuevo.
```

Figura 14: Salida incorrecta 3 ejercicio 4

- **Correo electrónico:** Dominio no permitido.
- **Contraseña:** Cumple con la longitud pero falta un carácter especial.

■ **Cuarto Intento:**

```
Correo electrónico: pablo@gmail.com
Contraseña: pablo123@

✅ Autenticación exitosa para el usuario pablo@gmail.com
PS C:\Users\ptl\Desktop\cuarto\ds\prac1\ej4> █
```

Figura 15: Salida exitosa ejercicio 4

- **Correo electrónico:** Formato y dominio correctos.
- **Contraseña:** Cumple con todos los requisitos.